

Análisis de complejidad computacional

Ciclos dependientes y sumatorias — Estructuras de Datos

Carlos A Delgado S

Pontificia Universidad Javeriana Cali

Agosto 2026

Objetivos de aprendizaje I

Al finalizar esta sesión, el estudiante podrá

- Contar ciclos anidados en los que las vueltas del ciclo interno dependen del índice externo.
- Reconocer el patrón de las vueltas a partir de una traza y escribirlo como una sumatoria.
- Cerrar sumatorias con las fórmulas de la suma de una constante y la suma de Gauss.

Objetivos de Aprendizaje del curso involucrados

- OA1: apropiar conocimientos de computación mediante el diseño e implementación de soluciones a problemas pequeños.
- OA3: describir ideas con argumentos precisos y basados en evidencia.

Objetivos de aprendizaje II

Referencia bibliográfica

T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein,
Introduction to Algorithms, 4.^a ed., sección 2.2 y apéndice A
(sección A.1, *Summation formulas and properties*); R. Thareja,
Data Structures Using C, capítulo 2.

Contenido I

- 1 El ejercicio pendiente
- 2 Ver el patrón
- 3 Fórmulas cerradas de las sumatorias
- 4 El conteo completo
- 5 Otras dependencias
- 6 Ejercicios
- 7 Resumen y referencias
- 8 Pistas de los ejercicios

Contenido I

- 1 El ejercicio pendiente
- 2 Ver el patrón
- 3 Fórmulas cerradas de las sumatorias
- 4 El conteo completo
- 5 Otras dependencias
- 6 Ejercicios
- 7 Resumen y referencias
- 8 Pistas de los ejercicios

Lo que quedó de la clase pasada I

- Cada instrucción simple cuenta 1 cada vez que se ejecuta.
- La condición de un `while` se evalúa una vez más que su cuerpo.
- Ciclos en secuencia suman su costo; ciclos anidados independientes lo multiplican.
- Cuando el conteo depende de los datos, se reporta el peor caso.

Y quedó un ejercicio sin resolver. Hoy es el punto de partida.

El ejercicio de cierre I

El ciclo interno depende del externo. Encuentre $T(n)$.

```
int triangulo(int n) {  
    int i = 0;  
    int cuenta = 0;  
    while (i < n) {  
        int j = 0;  
        while (j < i) {  
            cuenta = cuenta + 1;  
            j = j + 1;  
        }  
        i = i + 1;  
    }  
    return cuenta;  
}
```

¿Entendimos el problema? I

- **Entrada:** un entero $n \geq 0$.
- **Salida:** $T(n)$, el total de instrucciones que ejecuta la función.

Traza de `triangulo(4)`

Una fila por cada vuelta del ciclo externo:

i	Valores de j	Vueltas del cuerpo interno	cuenta
0	ninguno	0	0
1	0	1	1
2	0, 1	2	3
3	0, 1, 2	3	6

`triangulo(4)` devuelve 6: el propio `cuenta` lleva el registro de cuántas veces corrió el cuerpo interno.

Primer intento: multiplicar I

La clase pasada cerró con una regla: un ciclo adentro de otro multiplica.

- El ciclo externo da n vueltas.
- Si el interno diera n vueltas cada vez, el cuerpo se ejecutaría $n \cdot n$ veces.
- Con $n = 4$: cuenta debería terminar en $4 \cdot 4 = 16$.

Pero la traza dice otra cosa.

La traza contradice la cuenta I

multiplicar predice 16 la traza entrega 6.

El ciclo interno no hace siempre lo mismo: con $i = 0$ da 0 vueltas, con $i = 3$ da 3.

Multiplicar exige independencia

La regla “anidar multiplica” vale solo cuando el ciclo interno da la misma cantidad de vueltas en cada pasada. Si sus vueltas dependen del índice externo, multiplicar cuenta de más.

Contenido I

- 1 El ejercicio pendiente
- 2 Ver el patrón
- 3 Fórmulas cerradas de las sumatorias
- 4 El conteo completo
- 5 Otras dependencias
- 6 Ejercicios
- 7 Resumen y referencias
- 8 Pistas de los ejercicios

Las vueltas, una por una I

De la traza, el cuerpo interno corre:

i	Vueltas del cuerpo interno
0	0
1	1
2	2
3	3

$$\text{cuenta} = 0 + 1 + 2 + 3 = 6.$$

¿Cuánto vale cuenta para $n = 5$?

Una vuelta más I

Con $n = 5$ aparece una fila nueva y nada más cambia:

$$\text{cuenta} = 0 + 1 + 2 + 3 + 4 = 10.$$

¿Y para $n = 100$?

$$\text{cuenta} = 0 + 1 + 2 + \cdots + 99 = ?$$

Nadie quiere sumar cien términos a mano. Hace falta una forma corta de escribir la suma, y una fórmula que la resuelva.

El método para ciclos dependientes I

Congelar y sumar

Congele el índice externo i , cuente el ciclo interno como una función de i , y sume ese conteo sobre todas las vueltas del externo.

Aplicado de inmediato a la línea `cuenta = cuenta + 1;` con i congelado corre i veces, así que en total corre

$$0 + 1 + 2 + \cdots + (n - 1).$$

La notación para sumas largas I

La suma anterior se abrevia con el símbolo Σ :

$$\sum_{i=0}^{n-1} i = 0 + 1 + 2 + \cdots + (n - 1).$$

- Abajo va la variable y su valor inicial ($i = 0$).
- Arriba, el último valor que toma ($n - 1$).
- A la derecha, el término que se suma en cada paso.

El caso pequeño

$$\sum_{i=0}^3 i = 0 + 1 + 2 + 3 = 6.$$

Es la traza de `triangulo(4)` escrita en una línea.

La notación para sumas largas II

Una suma $\sum_{i=a}^b$ tiene $b - a + 1$ términos: los enteros de a a b , ambos incluidos.

Contenido I

- 1 El ejercicio pendiente
- 2 Ver el patrón
- 3 Fórmulas cerradas de las sumatorias**
- 4 El conteo completo
- 5 Otras dependencias
- 6 Ejercicios
- 7 Resumen y referencias
- 8 Pistas de los ejercicios

Sumar una constante I

Si el término no depende de la variable de la suma, sumar es multiplicar por el número de términos:

$$\sum_{i=a}^b k = k \cdot (b - a + 1).$$

Dos casos que aparecen al contar ciclos

$$\sum_{i=0}^{n-1} 1 = n \qquad \sum_{i=0}^{n-1} n = n \cdot n = n^2.$$

En la segunda, n es una constante para la suma: no cambia cuando i avanza. Por eso el resultado es n^2 y no n .

La pregunta de Gauss I

$$1 + 2 + 3 + \cdots + 100 = ?$$

Sin sumar los cien términos.

La suma, dos veces I

Se escribe la suma S de ida y de vuelta, alineadas:

$$\begin{array}{rcccccc} S & = & 1 & + & 2 & + & \cdots & + & 100 \\ S & = & 100 & + & 99 & + & \cdots & + & 1 \end{array}$$

¿Cuánto vale cada columna?

Cada columna vale lo mismo I

$$\begin{array}{rcccccc} S & = & 1 & + & 2 & + & \dots & + & 100 \\ S & = & 100 & + & 99 & + & \dots & + & 1 \\ \hline 2S & = & 101 & + & 101 & + & \dots & + & 101 \end{array}$$

Hay 100 columnas y cada una suma 101:

$$2S = 100 \cdot 101 = 10100 \qquad S = 5050.$$

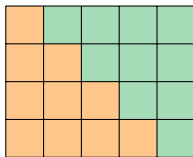
La suma de Gauss I

El mismo argumento con m en lugar de 100:

$$\sum_{i=1}^m i = \frac{m(m+1)}{2}.$$

Verificación con el caso más pequeño

$$1 + 2 + 3 = 6 \qquad \frac{3 \cdot 4}{2} = 6.$$



$$2S = 4 \cdot 5 \implies S = \frac{4 \cdot 5}{2} = 10$$

Dos escaleras de $1 + 2 + 3 + 4$ forman un rectángulo de 4×5 : la fórmula es este dibujo escrito en álgebra.

La versión que usa el conteo I

En los ciclos, i suele arrancar en 0 y terminar en $n - 1$. Se sustituye $m = n - 1$ (el 0 no aporta a la suma):

$$\sum_{i=0}^{n-1} i = \frac{(n-1)n}{2}.$$

Y la pregunta pendiente queda respondida:

$$\text{cuenta}(100) = \sum_{i=0}^{99} i = \frac{99 \cdot 100}{2} = 4950.$$

Las sumas se separan I

Dos reglas para desarmar una suma en sumas conocidas:

$$\sum_{i=a}^b (x_i + y_i) = \sum_{i=a}^b x_i + \sum_{i=a}^b y_i \qquad \sum_{i=a}^b k \cdot x_i = k \cdot \sum_{i=a}^b x_i.$$

Aplicada de inmediato

$$\sum_{i=0}^{n-1} (i + 1) = \sum_{i=0}^{n-1} i + \sum_{i=0}^{n-1} 1 = \frac{(n-1)n}{2} + n = \frac{n(n+1)}{2}.$$

Formulario I

Fórmulas cerradas (CLRS, apéndice A, sección A.1)

$$\sum_{i=a}^b k = k(b - a + 1) \quad \sum_{i=1}^m i = \frac{m(m+1)}{2} \quad \sum_{i=0}^{n-1} i = \frac{(n-1)n}{2}$$

$$\sum_{i=1}^m i^2 = \frac{m(m+1)(2m+1)}{6} \quad \sum_{i=0}^{n-1} i^2 = \frac{(n-1)n(2n-1)}{6}$$

Las dos primeras se dedujeron hoy; las de los cuadrados quedan de referencia para cuando un conteo las pida.

Contenido I

- 1 El ejercicio pendiente
- 2 Ver el patrón
- 3 Fórmulas cerradas de las sumatorias
- 4 El conteo completo**
- 5 Otras dependencias
- 6 Ejercicios
- 7 Resumen y referencias
- 8 Pistas de los ejercicios

triángulo, línea a línea I

En parejas: complete la columna de la derecha. Las líneas del ciclo interno se escriben como sumatorias sobre i .

Línea	Veces
<code>int i = 0;</code>	??
<code>int cuenta = 0;</code>	??
<code>while (i < n)</code>	??
<code>int j = 0;</code>	??
<code>while (j < i)</code>	??
<code>cuenta = cuenta + 1;</code>	??
<code>j = j + 1;</code>	??
<code>i = i + 1;</code>	??
<code>return cuenta;</code>	??

El conteo con sumatorias I

Línea	Veces
<code>int i = 0;</code>	1
<code>int cuenta = 0;</code>	1
<code>while (i < n)</code>	$n + 1$
<code>int j = 0;</code>	n
<code>while (j < i)</code>	$\sum_{i=0}^{n-1} (i + 1)$
<code>cuenta = cuenta + 1;</code>	$\sum_{i=0}^{n-1} i$
<code>j = j + 1;</code>	$\sum_{i=0}^{n-1} i$
<code>i = i + 1;</code>	n
<code>return cuenta;</code>	1

Con i congelado, el cuerpo interno da i vueltas y su condición se evalúa $i + 1$ veces: la evaluación que falla también cuenta, igual que la clase pasada.

Cerrar las sumas I

Cada sumatoria de la tabla se cierra con el formulario:

$$\sum_{i=0}^{n-1} (i+1) = \frac{(n-1)n}{2} + n = \frac{n(n+1)}{2}$$

$$\sum_{i=0}^{n-1} i = \frac{(n-1)n}{2} \quad (\text{dos veces: cuenta y j})$$

El total I

Se suma toda la columna:

$$\begin{aligned}T(n) &= 2 + (n + 1) + n + \frac{n(n + 1)}{2} + 2 \cdot \frac{(n - 1)n}{2} + n + 1 \\&= \frac{n(n + 1)}{2} + n(n - 1) + 3n + 4 \\&= \frac{n^2 + n + 2n^2 - 2n + 6n}{2} + 4 \\&= \frac{3n^2 + 5n}{2} + 4.\end{aligned}$$

La fórmula se pone a prueba I

Con $n = 2$, contando a mano

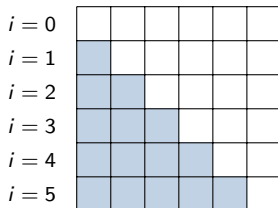
Inicializaciones: 2. Condición externa: 3. `int j = 0;`: 2. Condición interna: $1 + 2 = 3$. Cuerpo interno: $2 \cdot 1 = 2$. Incremento de i : 2. `return`: 1. Total: 15.

$$T(2) = \frac{3 \cdot 4 + 5 \cdot 2}{2} + 4 = \frac{22}{2} + 4 = 15.$$

La fórmula y la traza coinciden. Esa comprobación con un caso pequeño es parte del método: si no cuadra, alguna suma quedó mal cerrada.

El triángulo y el cuadrado I

Las vueltas del cuerpo interno, dibujadas: la fila i tiene i celdas.



$$\frac{n(n-1)}{2} \text{ celdas de } n^2$$

El costo del triángulo, $\frac{3n^2+5n}{2} + 4$, frente al del anidado completo de la clase pasada, $3n^2 + 4n + 4$:

n	triángulo	Anidado completo
10	179	344
100	15 254	30 404
1000	1 502 504	3 004 004

El triángulo y el cuadrado II

El triángulo hace la mitad del trabajo del cuadrado, pero crece igual: al multiplicar n por 10, ambos multiplican su costo por cerca de 100. La dependencia rebaja la constante, no el ritmo.

Contenido I

- 1 El ejercicio pendiente
- 2 Ver el patrón
- 3 Fórmulas cerradas de las sumatorias
- 4 El conteo completo
- 5 Otras dependencias**
- 6 Ejercicios
- 7 Resumen y referencias
- 8 Pistas de los ejercicios

El límite se corre en uno l

El mismo ciclo, con $\leq$ en lugar de $<$. En parejas: trace con $n = 4$ y diga cuántas veces corre el cuerpo interno.

```
int contar_incluida(int n) {  
    int i = 0;  
    int cuenta = 0;  
    while (i < n) {  
        int j = 0;  
        while (j <= i) {  
            cuenta = cuenta + 1;  
            j = j + 1;  
        }  
        i = i + 1;  
    }  
    return cuenta;  
}
```

El límite se corre en uno: respuesta I

i	Vueltas del cuerpo interno
0	1
1	2
2	3
3	4

$$\text{cuenta} = \sum_{i=0}^{n-1} (i+1) = \frac{n(n+1)}{2}$$

$$\text{contar_incluida}(4) = \frac{4 \cdot 5}{2} = 10.$$

El triángulo ahora incluye la diagonal: cada fila trae una celda más que en `triangulo`.

El ciclo interno arranca en i !

Sumar los productos de todas las parejas (i,j) con $i \leq j$.

```
int suma_parejas(int datos[], int n) {  
    int suma = 0;  
    int i = 0;  
    while (i < n) {  
        int j = i;  
        while (j < n) {  
            suma = suma + datos[i] * datos[j];  
            j = j + 1;  
        }  
        i = i + 1;  
    }  
    return suma;  
}
```

En parejas: congele i . ¿Cuántas vueltas da el ciclo interno como función de i ?

El patrón, al revés I

Con i congelado, j recorre $i, i + 1, \dots, n - 1$: son $n - i$ vueltas.

i	Vueltas del cuerpo interno ($n = 6$)
0	6
1	5
2	4
3	3
4	2
5	1

El patrón baja en lugar de subir. La suma se cierra con la linealidad:

$$\sum_{i=0}^{n-1} (n - i) = \sum_{i=0}^{n-1} n - \sum_{i=0}^{n-1} i = n^2 - \frac{(n-1)n}{2} = \frac{n(n+1)}{2}.$$

El patrón, al revés II

El mismo total de `contar_incluida`: es el mismo triángulo mirado desde el otro lado.

suma_parejas, línea a línea I

Línea	Veces
<code>int suma = 0;</code>	1
<code>int i = 0;</code>	1
<code>while (i < n)</code>	$n + 1$
<code>int j = i;</code>	n
<code>while (j < n)</code>	$\sum_{i=0}^{n-1} (n - i + 1)$
<code>suma = suma + datos[i] * datos[j];</code>	$\sum_{i=0}^{n-1} (n - i)$
<code>j = j + 1;</code>	$\sum_{i=0}^{n-1} (n - i)$
<code>i = i + 1;</code>	n
<code>return suma;</code>	1

suma_parejas: el total I

Las sumas ya están cerradas:

$$\sum_{i=0}^{n-1} (n-i) = \frac{n(n+1)}{2} \quad \sum_{i=0}^{n-1} (n-i+1) = \frac{n(n+1)}{2} + n.$$

$$\begin{aligned} T(n) &= 2 + (n+1) + n + \frac{n(n+1)}{2} + n + 2 \cdot \frac{n(n+1)}{2} + n + 1 \\ &= \frac{3n(n+1)}{2} + 4n + 4 = \frac{3n^2 + 11n}{2} + 4. \end{aligned}$$

La prueba del caso pequeño

Con $n = 1$: inicializaciones 2, condición externa 2, `int j = i; 1`, condición interna 2, cuerpo 2, incremento 1, `return 1`: total 11. Y la fórmula: $T(1) = \frac{3+11}{2} + 4 = 11$.

Contenido I

- 1 El ejercicio pendiente
- 2 Ver el patrón
- 3 Fórmulas cerradas de las sumatorias
- 4 El conteo completo
- 5 Otras dependencias
- 6 Ejercicios**
- 7 Resumen y referencias
- 8 Pistas de los ejercicios

Ejercicio 1 I

Complete la tabla línea a línea de `contar_incluida` y escriba $T(n)$. Las vueltas del cuerpo interno ya se contaron en la sesión.

Ejercicio 2 I

El límite interno es el doble del índice externo. ¿Cuántas veces corre el cuerpo interno? Escriba $T(n)$.

```
int doble_i(int n) {  
    int i = 0;  
    int cuenta = 0;  
    while (i < n) {  
        int j = 0;  
        while (j < 2 * i) {  
            cuenta = cuenta + 1;  
            j = j + 1;  
        }  
        i = i + 1;  
    }  
    return cuenta;  
}
```

Ejercicio 3 I

El externo salta de dos en dos y el interno depende de él. Suponga n par. ¿Cuántas veces corre el cuerpo interno?

```
int salto_dependiente(int n) {  
    int i = 0;  
    int cuenta = 0;  
    while (i < n) {  
        int j = 0;  
        while (j < i) {  
            cuenta = cuenta + 1;  
            j = j + 1;  
        }  
        i = i + 2;  
    }  
    return cuenta;  
}
```

Ejercicio de cierre I

El límite interno es el cuadrado del índice externo. Encuentre $T(n)$.

```
int cuadrado_interno(int n) {  
    int i = 0;  
    int cuenta = 0;  
    while (i < n) {  
        int j = 0;  
        while (j < i * i) {  
            cuenta = cuenta + 1;  
            j = j + 1;  
        }  
        i = i + 1;  
    }  
    return cuenta;  
}
```

Contenido I

- 1 El ejercicio pendiente
- 2 Ver el patrón
- 3 Fórmulas cerradas de las sumatorias
- 4 El conteo completo
- 5 Otras dependencias
- 6 Ejercicios
- 7 Resumen y referencias**
- 8 Pistas de los ejercicios

Lo que queda de la sesión

- Multiplicar vale solo si el ciclo interno es independiente; si depende del externo, se congela i , se cuenta el interno como función de i y se suma.
- La traza con un n pequeño revela el patrón de las vueltas; la sumatoria lo escribe en general.
- $\sum k = k(b - a + 1)$; $\sum_{i=1}^m i = \frac{m(m+1)}{2}$; las sumas se separan por linealidad.
- La fórmula final se comprueba contra la traza de un caso pequeño.
- El triángulo cuesta la mitad del cuadrado, pero ambos son cuadráticos: la dependencia baja la constante, no el ritmo de crecimiento.

Referencias I

-  T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein. *Introduction to Algorithms*. 4.^a ed., MIT Press, 2022. Sección 2.2 y apéndice A, sección A.1 (*Summation formulas and properties*).
-  R. Thareja. *Data Structures Using C*. Oxford University Press, 2018. Capítulo 2.

Contenido I

- 1 El ejercicio pendiente
- 2 Ver el patrón
- 3 Fórmulas cerradas de las sumatorias
- 4 El conteo completo
- 5 Otras dependencias
- 6 Ejercicios
- 7 Resumen y referencias
- 8 Pistas de los ejercicios**

Pistas I

- **Ejercicio 1:** las tres líneas internas ya quedaron contadas en la sesión; falta armar la tabla y sumar la columna. El resultado coincide con el de `suma_parejas`: los dos triángulos tienen el mismo tamaño.
- **Ejercicio 2:** con i congelado el cuerpo da $2i$ vueltas; el patrón es $0, 2, 4, 6, \dots$ y el 2 sale de la suma por linealidad:
$$\sum 2i = 2 \sum i.$$
- **Ejercicio 3:** solo los valores pares de i aportan. Con n par, i toma los valores $0, 2, \dots, n - 2$; escriba la suma de esos aportes y saque el 2 como factor para cerrar con Gauss.

El ejercicio de cierre se resuelve con el formulario completo de la sesión.